

LAPORAN TUGAS MODUL PRAKTIKUM 3
MATA KULIAH PENGEMBANGAN APLIKASI MOBILE



OLEH:
HARIS ALFARISY
NIM.245150401111009

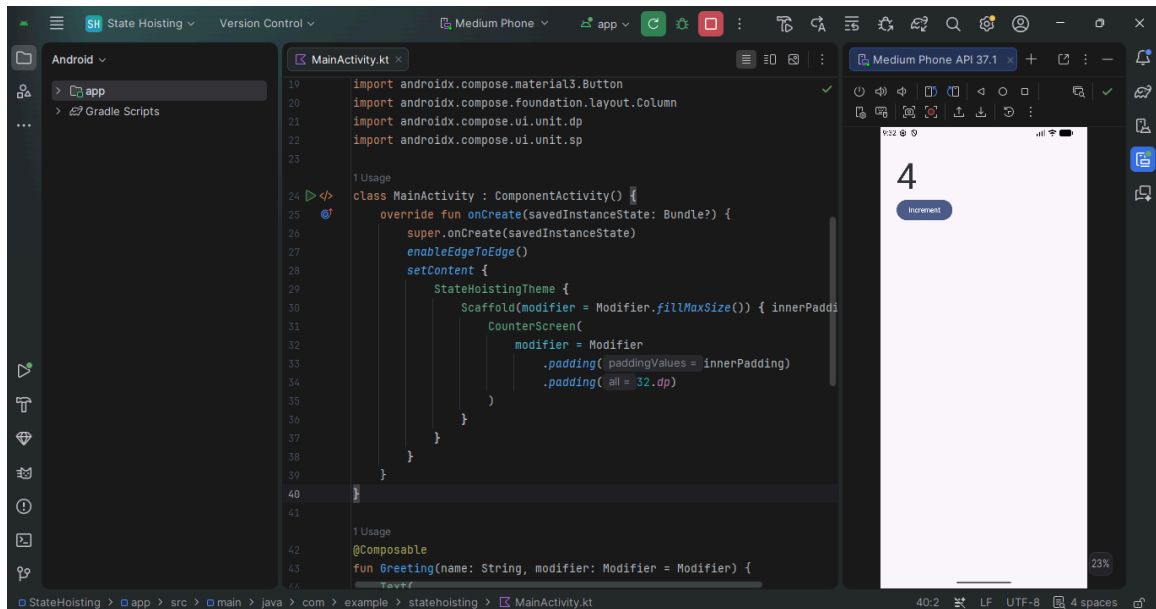
DOSEN PENGAMPU:
Muhammad Aminul Akbar, S.Kom., M.T.

SISTEM INFORMASI
DEPARTEMEN SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA
2026

A. State Hoisting

Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.



- a. Apa yang terjadi setiap kali tombol "Increment" diklik?
Jawab: yang terjadi yaitu jika tombol ditekan yaitu nilai yang awalnya 0 akan bertambah setiap di klik.
- b. Apa yang menyebabkan hal tersebut terjadi?
Jawab: hal tersebut terjadi dikarenakan adanya state number yang dideklarasikan dengan remember { mutableStateOf(0) }. Ketika tombol di klik maka lambda onClick = { number++ } dijalankan maka akan mengubah nilai state number yang memicu rekomposisi pada composable sehingga text menampilkan nilai baru.
- c. Variabel state apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?
Jawab: variabel state yang berperan yaitu number yang dideklarasikan dengan var number by remember { mutableStateOf(0) }
- d. Variabel state apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut?
Jawab: variabel state yang berperan yaitu label yang dideklarasikan dengan val label by remember { mutableStateOf("Increment") }

2. Pada percobaan tahap sebelumnya, ubah kode program pada Composable CounterScreen agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".

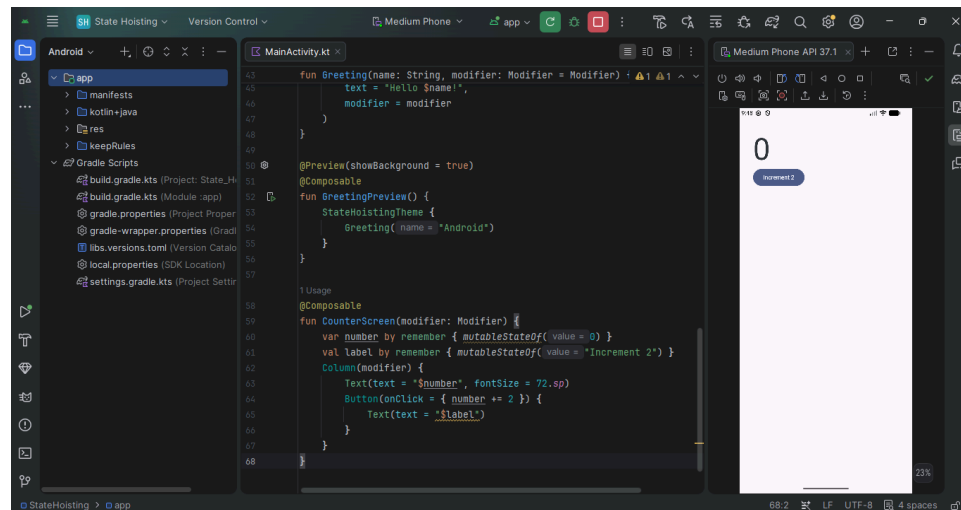
- a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?

Jawab: perubahan yang dilakukan ada di dalam composable CounterScreen pada lamda onClick Button ubah menjadi `number += 2` dan pada deklarasi state label ubah nilai menjadi "Increment 2"

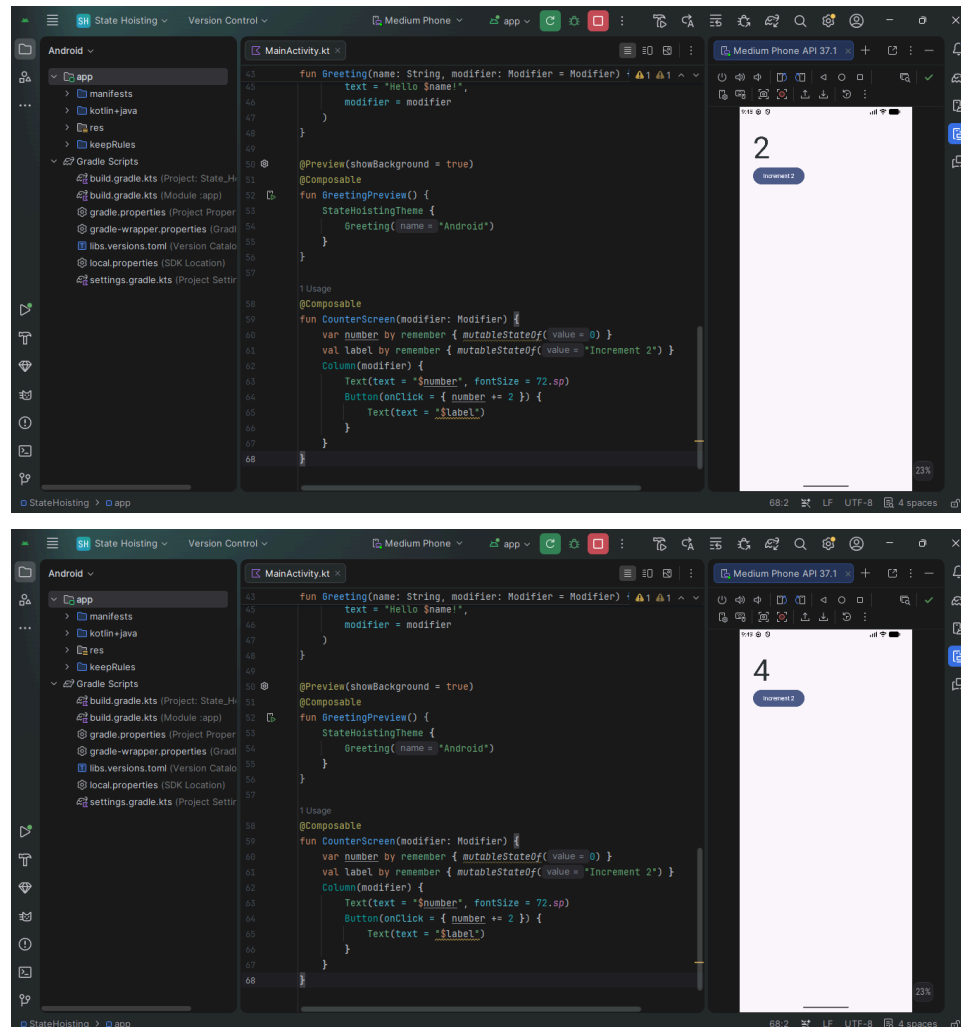
```
fun CounterScreen(modifier: Modifier) {  
    var number by remember { mutableStateOf(0) }  
    val label by remember { mutableStateOf("Increment 2") }  
    Column(modifier) {  
        Text(text = "$number", fontSize = 72.sp)  
        Button(onClick = { number += 2 }) {  
            Text(text = "$label")  
        }  
    }  
}
```

- b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.

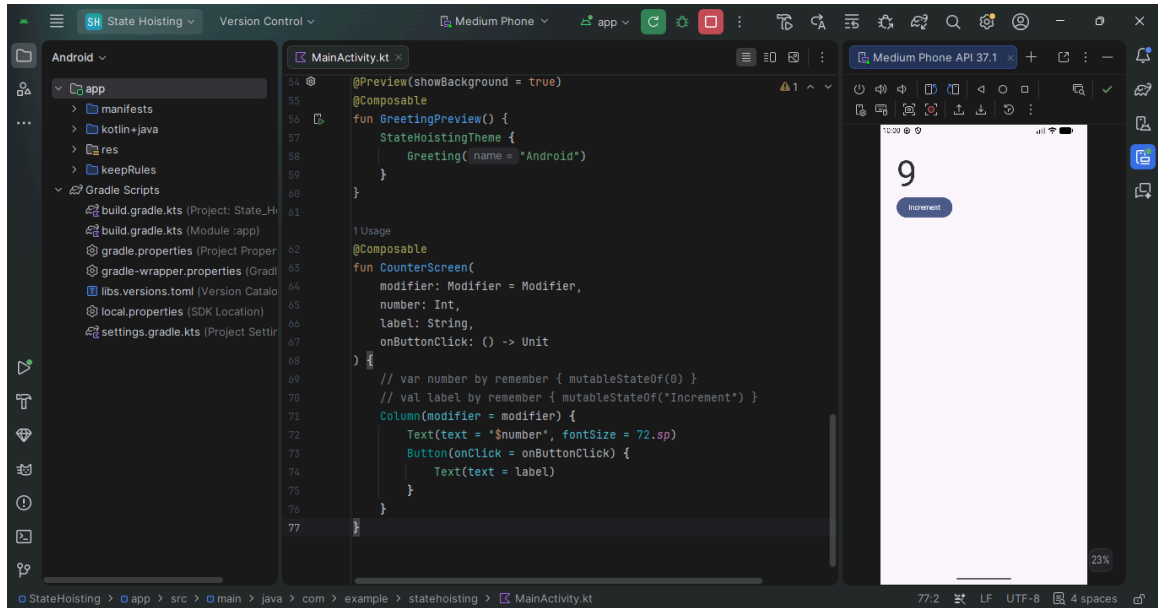
Sebelum diklik:



Sesudah diklik:



- Langkah selanjutnya adalah melakukan state dan event hoisting pada state dan event yang sebelumnya terdapat di dalam Composable CounterScreen. Ubah signature kode program pada bagian fungsi Composable CounterScreen. Perhatikan bahwa deklarasi variabel state `number` dari yang sebelumnya dilakukan di dalam fungsi Composable CounterScreen, sekarang dideklarasikan di dalam fungsi `onCreate()` sebelum fungsi Composable CounterScreen dipanggil dari dalam fungsi `onCreate()`.



4. Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.
 - a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?
Jawab: program berjalan dengan baik dan menampilkan hal yang sama persis, angka tetap bertambah 1 setiap kali tombol diklik dan label tetap "Increment"
 - b. Di bagian kode program manakah saat ini proses increment dilakukan?
Jawab: increment dilakukan di dalam fungsi onCreate pada lamda onClick yang dikirim sebagai parameter ke composable
 - c. Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan?
Jawab: state number dideklarasikan di dalam fungsi onCreate, tidak di dalam composable. Composable CounterScreen bersifat stateless, hanya menerima nilai number dan label melalui parameter
5. Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna. Ubah kode program setelah tahap nomor 7 dilakukan sedemikian rupa sehingga:
 - a. Label tombol "Increment" menjadi "Double";
 - b. Nilai awal variabel number dari 0 menjadi 1.
 - c. Ubah fungsi/lambda onClick pada pemanggilan Composable CounterScreen di dalam fungsi onCreate() sedemikian rupa sehingga yang dilakukan dari sebelumnya: number++ menjadi menggandakan nilai variabel number.

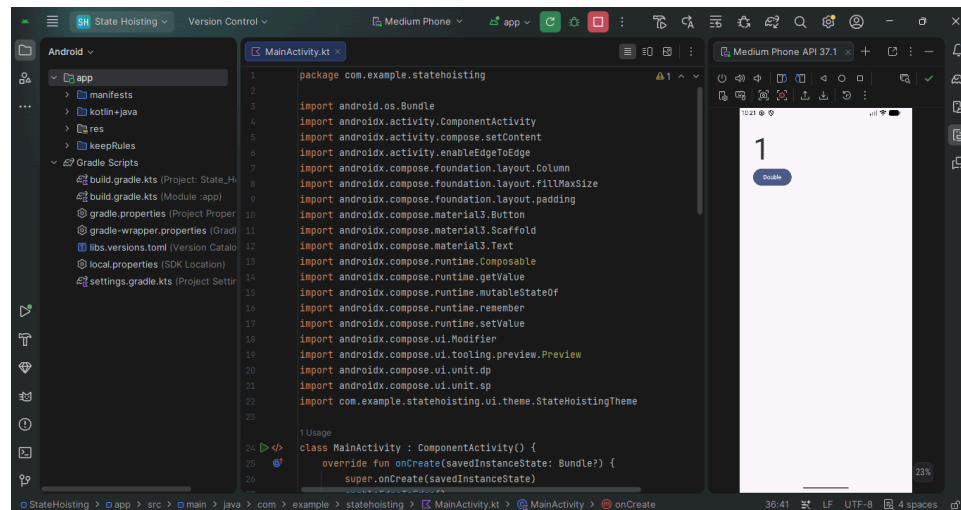
- d. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi Composable CounterScreen di dalam fungsi onCreate() setelah langkah nomor 5(a), 5(b), dan 5(c) dilakukan.

```
var number by remember { mutableStateOf(1) }  
CounterScreen(  
    Modifier.padding(32.dp),  
    number = number,  
    label = "Double",  
    onClick = { number *= 2 }  
)
```

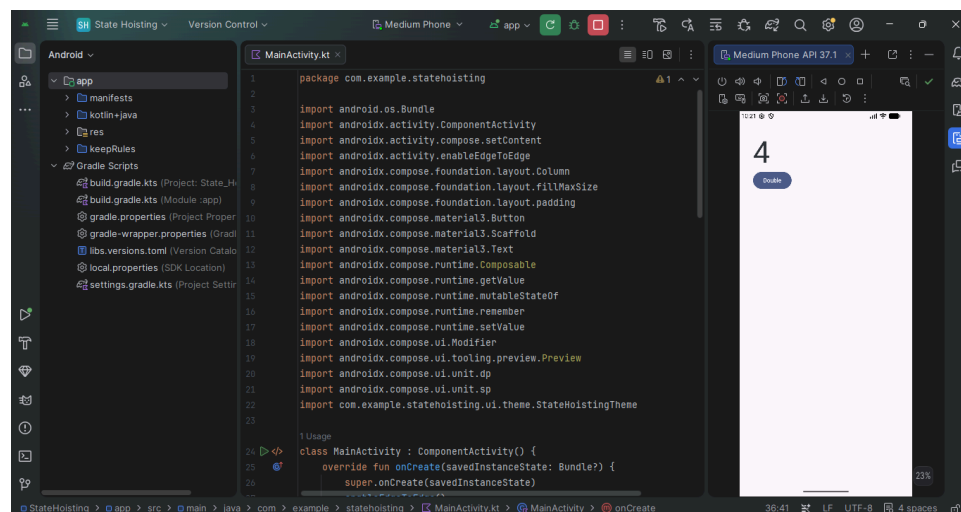
6. Jalankan program,

- a. Perlihatkan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel number digandakan.

Sebelum:



Sesudah:



- b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?

Jawab: percobaan tahap 7 dilakukan dalam fungsi onCreate dan di dalam signature composable CounterScreen sedangkan percobaan tahap 8 dilakukan dalam fungsi onCreate pada deklarasi state number dan pemanggilan CounterScreen.

- c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri? Jika ada sebutkan perubahan yang dilakukan.

Jawab: tidak ada perubahan yang dilakukan di dalam fungsi Composable CounterScreen itu sendiri pada tahap nomor 8.

7. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan state hoisting dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?

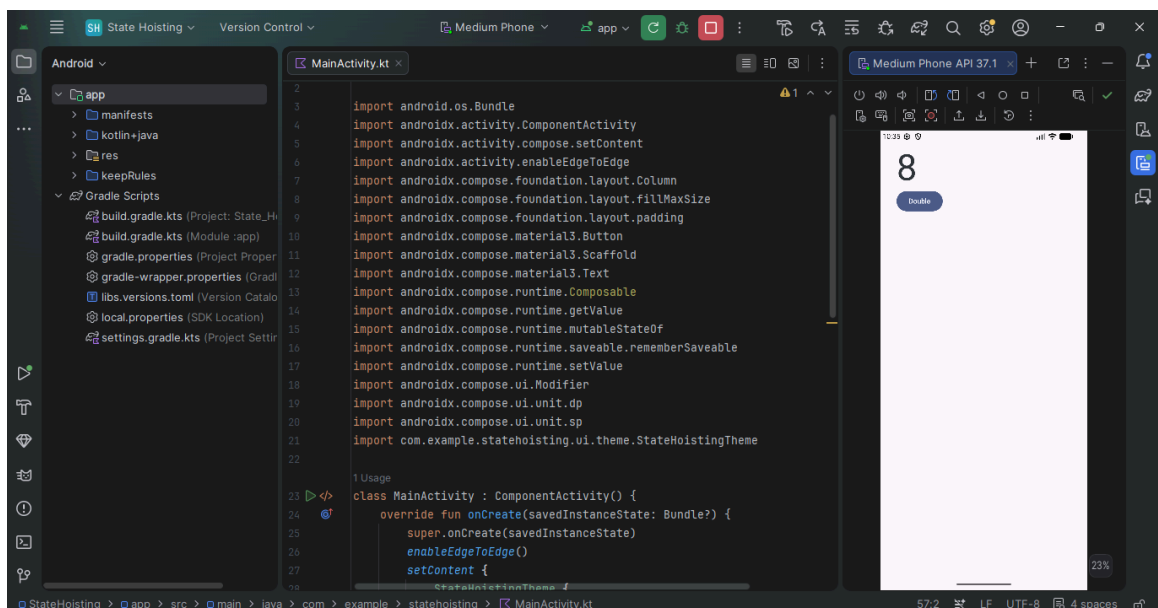
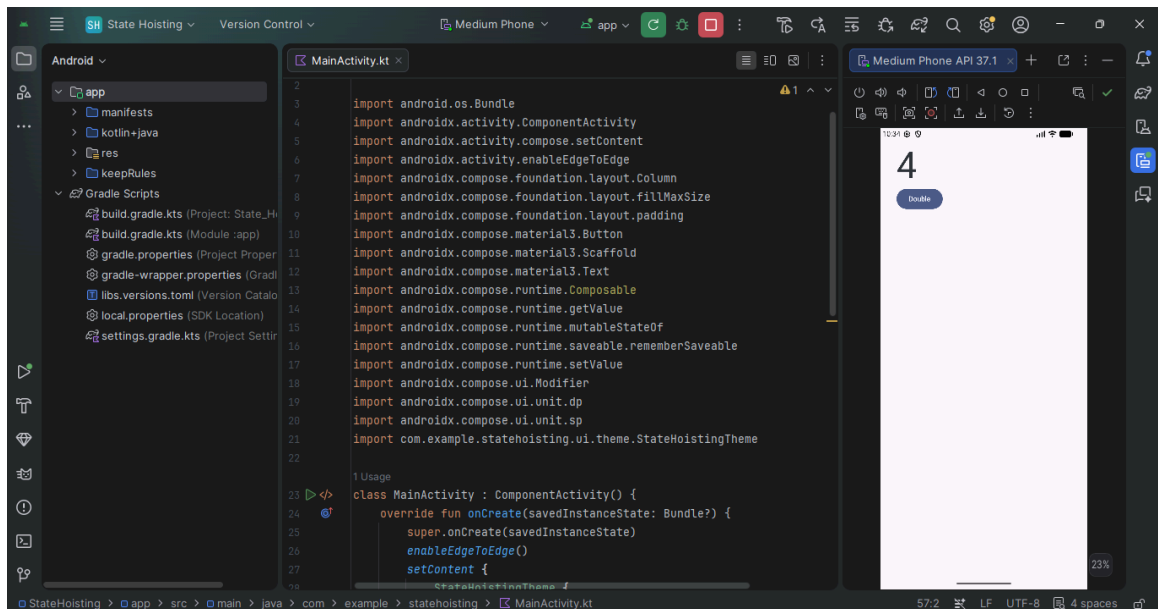
Jawab: Jawaban pada tahap nomor 9 menunjukkan bahwa dengan state hoisting, perubahan kebutuhan fungsional seperti mengubah nilai awal, label tombol, dan logika increment dapat dilakukan hanya dengan mengubah kode di satu tempat, yaitu di komponen induk (onCreate()), tanpa perlu menyentuh Composable CounterScreen sama sekali.

B. Perubahan Konfigurasi

Aktivitas Percobaan

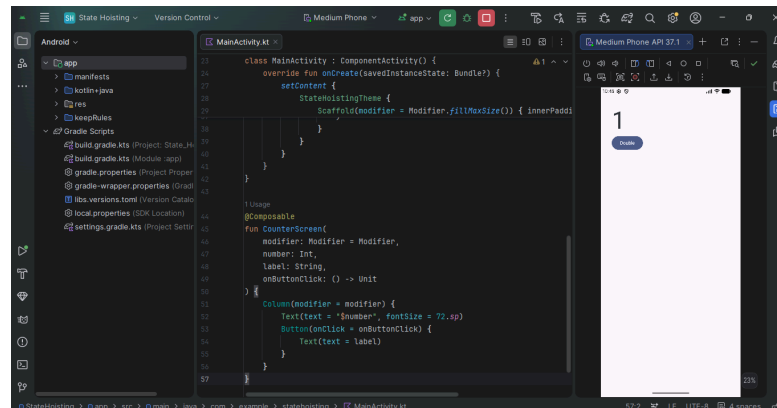
1. Jalankan program, dengan memastikan posisi perangkat pada posisi portrait, kemudian klik beberapa kali tombol "Double"; sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan.

Catat angka yang ditampilkan setelah tombol "Double"; diklik beberapa kali, ambil screenshot tampilan antarmukanya.



2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.

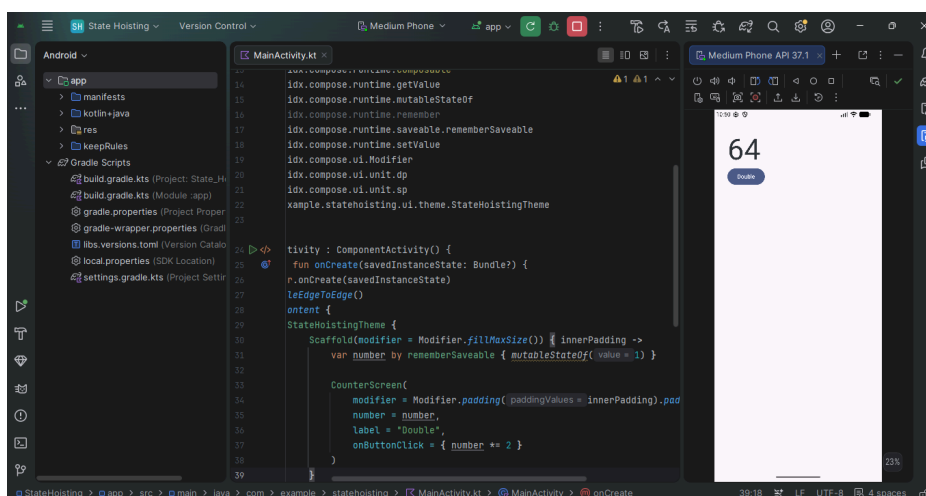
Nilai angka yang tercatat saat posisi landscape adalah 1



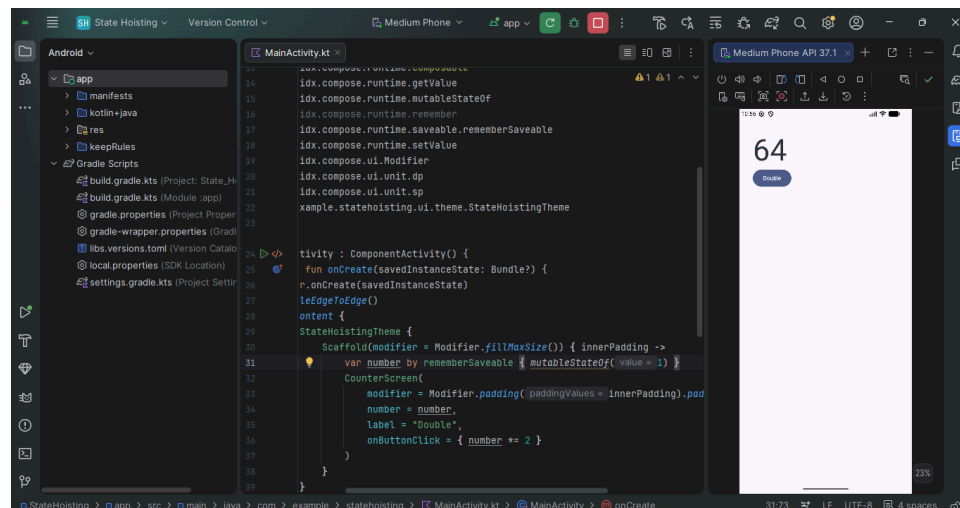
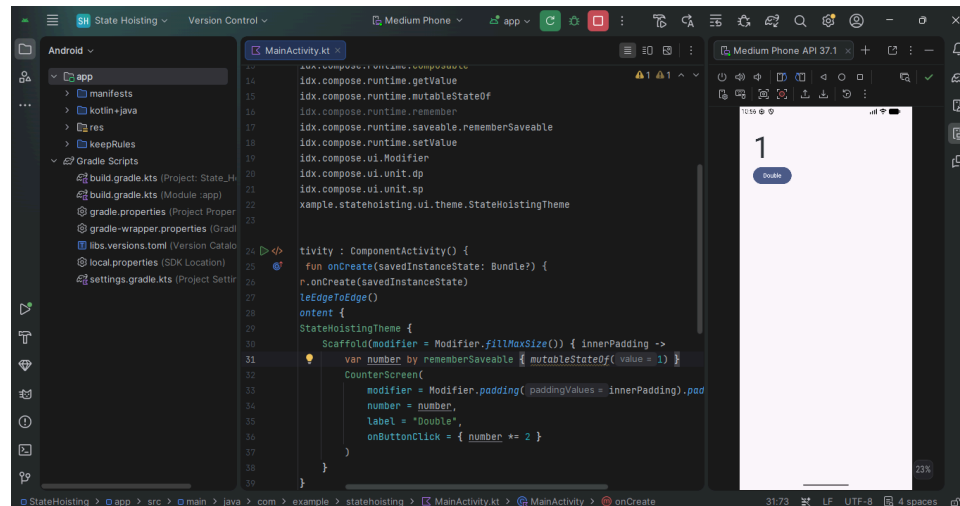
- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Jawab: Angka kembali menjadi 1 karena saat layar diputar, sistem Android menghancurkan dan membuat ulang Activity dari awal (configuration change). Penggunaan konstruk remember hanya bisa menjaga nilai saat tampilan dimuat ulang (recomposition), bukan saat Activity dibuat ulang. Akibatnya, data lama terhapus dari memori dan nilainya ter-reset kembali ke angka awal yaitu 1.

Compile dan jalankan kembali program, dengan memastikan posisi perangkat pada posisi portrait, kemudian klik beberapa kali tombol "Double"; sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan. Catat angka yang ditampilkan setelah tombol "Double"; diklik beberapa kali, ambil screenshot tampilan antarmukanya.



3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi portrait menjadi landscape, sehingga tampilan aplikasi akan disesuaikan oleh Android.
 - a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



- b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Jawab: angka yang ditampilkan berupa nilai terakhir, tidak kembali ke angka 1 dikarenakan rememberSaveable menyimpan nilai state ke dalam objek Bundle (saved instance state). Meskipun sistem Android menghancurkan dan membuat ulang Activity saat layar diputar, nilai data tersebut secara otomatis dipulihkan kembali dari Bundle sehingga angkanya tidak ter-reset.

- c. Apa fungsi dari konstruk rememberSaveable pada kode program Android yang membedakannya dari konstruk remember?

Jawab: rememberSaveable menjaga nilai state baik saat rekomposisi maupun saat terjadi perubahan konfigurasi (rotasi layar) hingga penghentian proses oleh sistem (process death), karena datanya disimpan secara aman di dalam Bundle, sedangkan remember hanya menjaga nilai state selama proses rekomposisi antarmuka.

4. Beri contoh kasus nyata lain di mana penggunaan konstruk rememberSaveable ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

Jawab: contoh kasus nyata seperti formulir pendaftaran atau pengisian data panjang dimana skenario nya pengguna sedang mengisi formulir panjang seperti pendaftaran akun, penerapannya menggunakan rememberSaveable untuk menyimpan string input teks. Jika pengguna tidak sengaja memutar ponsel atau sistem beralih ke mode multi-window/split-screen, teks yang sudah diketik panjang lebar tidak akan hilang.

C. Launched Effect

Aktivitas Percobaan

1. Jalankan program,

- a. Pada saat pertama kali program dijalankan, tanpa mengklik button "Double", apa yang ditampilkan di dalam layar?

Jawab: yang ditampilkan yaitu angka 1 bersama tombol button yang bertuliskan "Double" dan pada bagian bawah layar terdapat notifikasi Snackbar dengan pesan teks yaitu Number 1 is shown!

- b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?

Jawab: pesan tersebut ditampilkan dikarenakan sifat dasar dari LaunchedEffect yaitu meluncurkan blok coroutine miliknya saat pertama kali composable masuk ke dalam komposisi, karena state awal variabel number bernilai 1, LaunchedEffect langsung mengeksekusi blok kode di dalamnya saat layar pertama kali dimuat.

- c. Apa yang memicu pesan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?

Jawab: pemicu pesan ditampilkan yaitu inisialisasi composable pertama kali ke dalam hirarki UI, bagian kode nya yaitu pada blok LaunchedEffect di dalam Scaffold:

```
LaunchedEffect(number) {  
    snackbarHostState.showSnackbar("Number $number is shown!")  
}
```

2. Klik button "Double", satu kali,

- a. Apa yang terjadi?

Jawab: jika button double di klik satu kali maka angka akan berubah menjadi 2 dan notifikasi snackbar akan muncul dengan kata yaitu Number 2 is shown!

- b. Mengapa hal tersebut terjadi? Jelaskan!

Jawab: ketika button double di klik maka aksi pada onClick akan mengubah nilai state number menjadi 2 yang langsung memicu proses rekomposisi UI untuk memperbarui tampilan teks angka di layar. Karena nilai parameter kunci pada LaunchedEffect berubah maka coroutine yang lama dibatalkan dan digantikan oleh yang baru yang kemudian mengeksekusi snackbarHostState sehingga notifikasi Snackbar otomatis diperbarui dengan teks angka yang terkini.

Aktivitas Percobaan

1. Jalankan program,

- a. Pada saat pertama kali program dijalankan, tanpa mengklik button "Double"; apa yang ditampilkan di dalam layar?

Jawab: yang ditampilkan di layar utama yaitu angka awal 1 dengan tombol button bertuliskan "double" lalu dibagian bawah terdapat notifikasi yang menampilkan pesan teks yaitu "Number 1 is shown!"

- b. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?

Jawab: setelah menunggu selama 2 detik, maka angka pada layar berganti secara otomatis dari angka 1 menjadi angka random tiga digit dan pada bagian bawah muncul notifikasi snackbar dengan angkanya menyesuaikan angka yang muncul tiga digit di atas.

- c. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?

Jawab: pemicu perubahan angka yaitu selesainya eksekusi fungsi suspend `Repo.getData()` setelah penundaan simulasi jaringan selama 2000 ms (`delay(2000)`), yang kemudian menimpa state number dengan nilai acak baru. Pemicu perubahan pesan snackbar yaitu reaktivitas terhadap perubahan nilai state number. Karena parameter kunci (key) pada `LaunchedEffect(number)` mendeteksi adanya perubahan nilai dari 1 menjadi angka acak yang baru, blok efek lama dibatalkan dan dijalankan kembali untuk menampilkan Snackbar terbaru. Bagian kode nya yaitu:

```
LaunchedEffect(number) {  
    snackbarHostState.showSnackbar("Number $number is shown!")  
}
```

2. Dari kedua percobaan terkait dengan side-effect dengan `LaunchedEffect`,

- a. Jelaskan apa perbedaan antara `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)`?

Jawab: perbedaan utamanya terletak pada sifat parameter kunci (key) yang digunakan: `LaunchedEffect(state)` bersifat reaktif karena menggunakan variabel data dinamis yang akan membatalkan coroutine lama dan meluncurkan coroutine baru setiap kali nilai state tersebut berubah (cocok untuk sinkronisasi data atau notifikasi berulang), sedangkan `LaunchedEffect(Unit)` menggunakan objek konstan yang nilainya tidak pernah berubah, sehingga coroutine di dalamnya hanya dieksekusi sekali saat pertama kali masuk ke komposisi tanpa terpengaruh

oleh rekomposisi berikutnya (sangat ideal untuk initial setup seperti memuat data API atau database saat layar dibuka).

- b. Kapan operasi di dalam `LaunchedEffect(state)` dengan `LaunchedEffect(Unit)` dijalankan?

Jawab: operasi di dalam `LaunchedEffect(Unit)` hanya dijalankan satu kali saat `Composable` pertama kali masuk ke dalam komposisi (`initial composition`) dan tidak akan diulang kembali meskipun UI mengalami proses rekomposisi berkali-kali, sedangkan operasi di dalam `LaunchedEffect(state)` dijalankan saat pertama kali masuk ke komposisi dan akan dieksekusi ulang secara otomatis setiap kali nilai variabel `state` yang menjadi parameter kuncinya mengalami perubahan (dengan membatalkan `coroutine` lama yang sedang berjalan terlebih dahulu).